

Hackathon — Sistema de Processamento de Vídeos

Arquitetura de Software — FIAP Pós-Tech SOAT

RM: 369371 | Grupo: 22 | Turma: 12SOAT | Aluna: Nicole Bono Martins

1. Repositório e Serviços

Repositório (monorepo): github.com/NicoleBono/fiapx-video-hackathon

Serviço	Tipo	Responsabilidade	Persistência
auth-service	HTTP :3001	Registro, login e emissão de JWT	PostgreSQL (auth_db)
video-service	HTTP :3000	Upload, listagem de status e download do zip	PostgreSQL (video_db) + Redis (cache) + MinIO (arquivos)
processing-service	worker	Consome a fila, extrai frames (ffmpeg) e zipa. Stateless e escalável horizontalmente.	MinIO (arquivos)
notification-service	worker	Consome eventos de falha e envia e-mail de aviso ao usuário	—

2. Vídeo de Demonstração

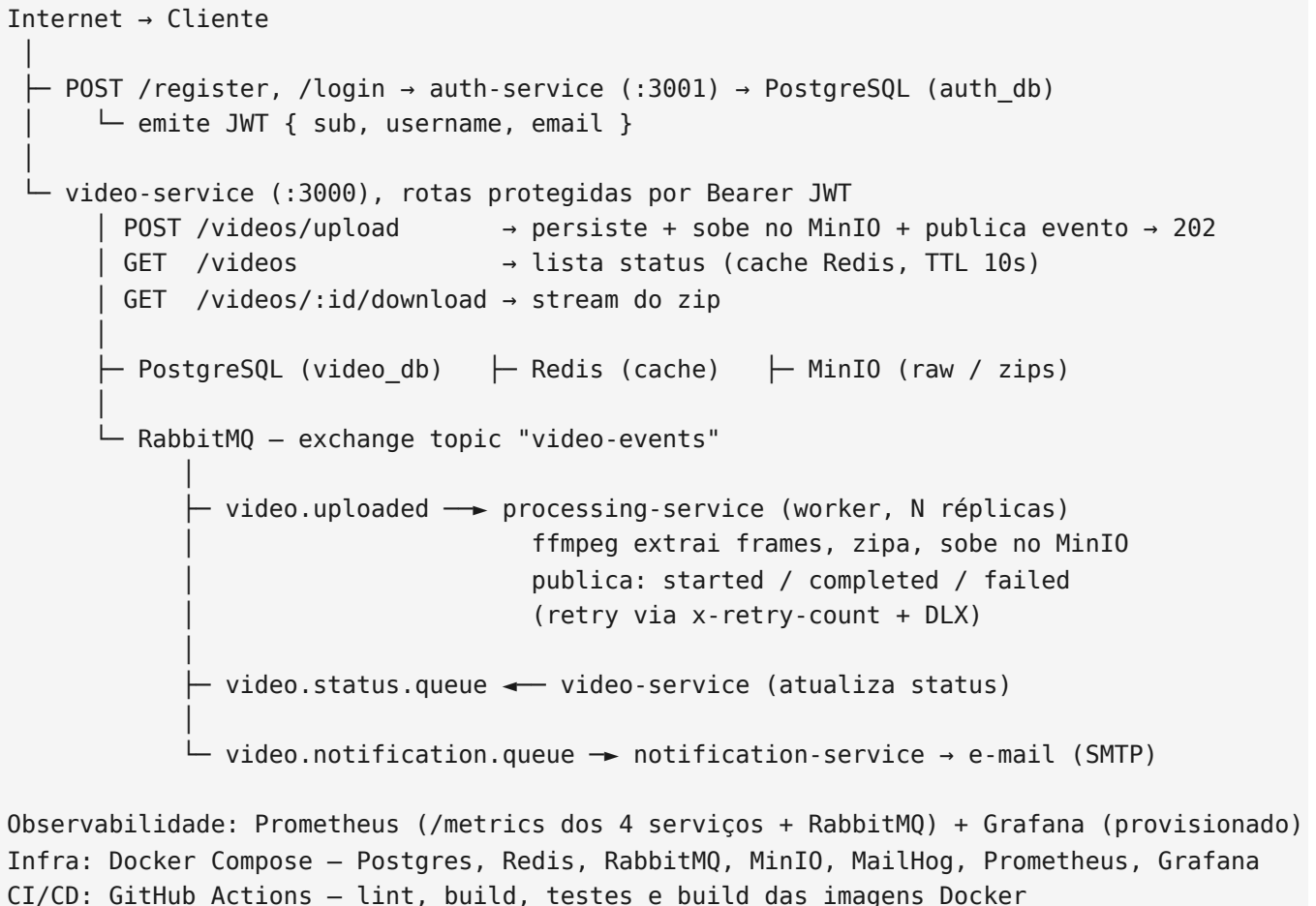
Link do vídeo: https://www.youtube.com/watch?v=kwxl_oDH53I

O vídeo demonstra:

- Documentação e arquitetura escolhida
- Fluxo completo: registro → login → upload de vídeo → acompanhamento de status → download do zip de frames
- Tratamento de falha no processamento com notificação automática por e-mail
- Processamento de múltiplos vídeos simultaneamente (RabbitMQ distribuindo entre réplicas do processing-service)

- Pipeline de CI/CD no GitHub Actions
- Monitoramento em tempo real com Prometheus + Grafana

3. Diagrama Geral da Arquitetura



4. Mensageria — Estratégia Escolhida: Eventos Assíncronos via RabbitMQ (topic exchange)

Justificativa

Foi adotado um fluxo **assíncrono orientado a eventos** via RabbitMQ, com uma única exchange topic (video-events) e filas dedicadas por consumidor. Essa abordagem foi escolhida porque:

- Desacopla o upload do processamento: o video-service responde 202 assim que persiste o metadado e publica o evento — nenhuma requisição é perdida em pico, porque a fila absorve o excesso

- Fan-out nativo da exchange topic: o mesmo evento (`video.failed`) é consumido por dois serviços diferentes (`video-service` e `notification-service`) sem acoplamento entre eles
- `processing-service` é totalmente stateless — qualquer número de réplicas compete pela mesma fila (`prefetch=1`), escalando horizontalmente sem nenhuma coordenação entre instâncias
- Retry automático controlado por um header na própria mensagem (`x-retry-count`), com uma Dead Letter Exchange como rede de segurança para o caso do worker cair sem nem dar nack

Fluxo principal — Upload e Processamento

1. Cliente → POST `/videos/upload` (Bearer JWT)
`video-service` persiste `Video` (`status: PENDING`), sobe o arquivo bruto no MinIO
↳ publica: `video.uploaded { videoId, userId, email, objectKey, originalFilename }`
Responde 202 imediatamente (upload não bloqueante)
2. `processing-service` consome `video.uploaded`
↳ publica: `video.processing.started { videoId }`
`video-service` consome → atualiza `status: PROCESSING`
3. `processing-service` baixa o raw, extrai frames (ffmpeg), zipa e sobe no MinIO
↳ publica: `video.completed { videoId, frameCount, zipObjectKey }`
`video-service` consome → atualiza `status: DONE`
4. Cliente → GET `/videos/:id/download` → stream do zip de frames

Tratamento de falha

`processing-service` falha ao processar (ex: arquivo inválido)
↳ republica com `x-retry-count + 1` (até `RABBITMQ_MAX_RETRIES`, default 3)

Esgotadas as tentativas:
↳ publica: `video.failed { videoId, userId, email, errorMessage, permanent: true }`

`video-service` consome → atualiza `status: FAILED` ✓
`notification-service` consome → envia e-mail de aviso ao usuário via SMTP ✓

5. Justificativa da Divisão dos Serviços e Tecnologias

auth-service — PostgreSQL

Isolado dos demais: é o único que conhece credenciais e hash de senha (bcrypt). Emite um JWT autocontido (`{sub, username, email}`) que os outros serviços validam localmente, sem round-trip síncrono a cada requisição.

video-service — PostgreSQL + Redis + MinIO

Único ponto de contato HTTP do usuário para vídeo. Persiste metadado num banco isolado (video_db, sem acesso direto ao auth_db), usa Redis como cache de leitura (cache-aside, TTL 10s, invalidado a cada upload ou mudança de status) para a listagem não bater sempre no banco, e MinIO (S3-compatible) para o arquivo bruto e o zip — desacopla o armazenamento dos containers de processamento e permite múltiplas réplicas sem volume compartilhado.

processing-service — worker stateless, sem banco

Não guarda nenhum estado em memória nem em disco além de um diretório temporário por job; qualquer número de réplicas consome a mesma fila em paralelo. Usa `fluent-ffmpeg` para extração de frames e `archiver` para compactação.

notification-service — worker stateless, sem banco

Consome apenas o evento de falha e dispara o e-mail via SMTP; fica desacoplado do fluxo principal para que um problema no envio de e-mail nunca trave o processamento de vídeos.

Tecnologias transversais

Tecnologia	Uso	Justificativa
NestJS 10 + TypeORM	Todos os serviços	Framework modular com injeção de dependência nativa; mesmo padrão em toda a stack
RabbitMQ (amqplib)	Mensageria	Exchange topic com fan-out nativo, DLX e retry simples de implementar, fácil de rodar via Docker
MinIO	Armazenamento de arquivos	Compatível com S3 — troca fácil por um bucket real em produção sem mudar código
Redis (ioredis)	Cache	Cache-aside simples para a listagem de status, sem sobrecarregar o Postgres
Prometheus + Grafana	Monitoramento	Métricas de negócio e infraestrutura (RabbitMQ nativo via plugin), dashboard provisionado automaticamente
Docker Compose	Orquestração local	Escala horizontal do processing-service via <code>--scale</code> , suficiente para o escopo do desafio
GitHub Actions	CI/CD	Pipeline único cobrindo os 4 serviços do monorepo

6. Testes e Qualidade

Serviço	Testes unitários	Testes e2e	Cobertura de linhas (aprox.)
auth-service	10	6	~90%
video-service	33	5	~94%
processing-service	17	—	~90%
notification-service	13	—	~90%

Total: **73 testes unitários + 11 testes e2e = 84 testes**. Os testes unitários mockam toda a infraestrutura (RabbitMQ, MinIO, Redis, SMTP, repositórios TypeORM), e os e2e sobem a aplicação real via supertest contra o guard/pipe/controller reais — nenhum deles depende de Postgres, Redis, RabbitMQ ou MinIO estarem no ar, então rodam em CI sem infraestrutura externa.

7. CI/CD

Pipeline único no GitHub Actions (`.github/workflows/ci.yml`), cobrindo os 4 serviços do monorepo:

```
on: push (main) | pull_request (main) | workflow_dispatch

jobs:
  test (matriz: auth-service, video-service, processing-service, notification-service)
    → npm ci → lint --if-present → build → testes unitários (--coverage) → e2e (auth/video)

  compose-validate
    → docker compose config -q

  docker-build (matriz, depende de "test")
    → build das 4 imagens Docker (buildx + cache do GitHub Actions)
```

8. Monitoramento

Prometheus e Grafana provisionados automaticamente via Docker Compose — datasource e dashboard já vêm prontos, sem configuração manual:

- auth-service e video-service expõem GET /metrics na própria porta HTTP

- processing-service e notification-service (workers, sem servidor HTTP) sobem um mini servidor só para /metrics, na rede interna do compose
- RabbitMQ expõe métricas nativas via plugin rabbitmq_prometheus
- Dashboard "FIAP X - Visão Geral" com 9 painéis: taxa e latência (p95) de requisições HTTP, uploads/downloads de vídeo por minuto, eventos de status consumidos, jobs de processamento por resultado, notificações enviadas, profundidade das filas do RabbitMQ e memória por processo